ETIB

Interpreter Self-Training Platform

Ecole de Traducteurs et d'Interpretes de Beyrouth | USJ Beirut

Technical Documentation

Developer Reference — Architecture, Setup & API

Working languages: Arabic | French | English

Summer 2026 internship - ESIB, ETIB, CINIA, Universite Saint-Joseph

Supervised by Prof. Lina Sader Feghali and Prof. Wadad Wazen Gergy

Generated November 07, 2026

Project Overview

The ETIB Interpreter Self-Training Platform is an AI-powered web application that lets trainee conference interpreters practice independently. Students generate realistic training speeches, listen to them via text-to-speech, record their own interpretation, and receive instant automated feedback — all in one browser session.

Module	What it produces
A Speech Generation	Configurable training speech (topic, domain, length, difficulty, hesitations, number density, discourse structure) grounded in real UN documents when available
B Audio & Materials	Edge-TTS audio with accent/speed + key terms, thematic summary, MCQ, comprehension questions, trilingual glossary (AR/FR/EN), downloadable DOCX
C Transcription	Groq Whisper-large-v3 ASR (falls back to local faster-whisper) + Arabic tashkeel reflecting what the student actually said
D Evaluation	Hesitation count, omission detection, number errors, terminology coverage, pronunciation alignment, LLM feedback paragraph, adaptive recommendations
E Progress	Session history, score trend (last 10 sessions), focus areas, strengths, specific recurring errors

Architecture

```
Browser (React + Vite, hosted on Vercel)
  |  fetch() -- JSON bodies, auth token in the body
  v
Flask backend  (Python 3.11, Docker on Hugging Face Spaces)
  +-- /api/module-a   -->  LLM speech generation  (llm_service.py -> Gemini)
  +-- /api/module-b   -->  TTS (edge-tts) + materials (Gemini)
  +-- /api/module-c   -->  ASR transcription (Groq Whisper / faster-whisper)
  +-- /api/module-d   -->  Evaluation + feedback (Gemini) + sessions (MongoDB)
  +-- /api/library    -->  UN Digital Library search + PDF download
  +-- /api/chat       -->  Chat endpoint
  +-- /api/auth       -->  Login / signup
```

Central server-side keys: the backend holds the API keys (Gemini for text generation and evaluation, Groq for Whisper ASR). Students do not need an API key of their own, and no key is stored in or sent from the browser.

Tech Stack

Layer	Technology
Frontend	React 18, Vite, plain CSS (no UI framework)
Backend	Flask 3, Python 3.11
LLM	Google Gemini via REST API (default gemini-3.5-flash-lite); Groq and local Aya supported
ASR	Groq hosted whisper-large-v3 — falls back to local faster-whisper
TTS	edge-tts (Microsoft Azure Neural voices, free)
Embeddings	paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers) for RAG
Database	MongoDB (local) with in-memory fallback for development
UN Library	UN Digital Library MARCXML API + curl PDF download
Deployment	Vercel (frontend), Hugging Face Spaces Docker (backend)

Local Setup

Prerequisites

- Python 3.10+
- Node.js 18+
- ffmpeg (winget install ffmpeg / brew install ffmpeg / sudo apt install ffmpeg)
- MongoDB — optional (app runs without it; sessions are stored in memory)

1 — Clone and configure

```
git clone https://github.com/joe-haddad3/etib-interpreter-trainer.git
cd etib-interpreter-trainer
```

Copy backend/.env.example to backend/.env and fill in at least:

```
LLM_PROVIDER=gemini
GOOGLE_AI_KEY=your_google_ai_key_here   # Gemini
GROQ_API_KEY=your_groq_api_key_here     # Whisper ASR
FLASK_SECRET_KEY=change-me-in-production
FLASK_DEBUG=true
UPLOAD_FOLDER=./uploads
AUDIO_OUTPUT_FOLDER=./audio_outputs
# MONGODB_URI is optional; without it, sessions are kept in memory
```

2 — Backend

```
cd backend
python -m venv venv
venv\Scripts\activate        # Windows
# source venv/bin/activate    # macOS / Linux
pip install -r requirements.txt
python app.py
# Server starts at http://127.0.0.1:5000
```

3 — Frontend

```
cd Frontend
npm install
npm run dev
# Opens at http://localhost:5173
```

API Keys

The platform runs on central server-side keys, set as environment variables on the backend (GOOGLE_AI_KEY for Gemini, GROQ_API_KEY for Whisper ASR). They are never stored in the repository. Students sign in with an account; they never handle API keys.

Free-tier quotas of both providers change over time, so check the current limits on Google AI Studio and console.groq.com before load testing.

Project Structure

```
etib-interpreter-trainer/
|-- backend/
|   |-- app.py               Flask app, blueprints, CORS
|   |-- config.py            All environment variables and constants
|   |-- requirements.txt
|   |-- modules/
|   |   |-- module_a.py      Speech generation (LLM + RAG + UN grounding)
|   |   |-- module_b.py      TTS + pedagogical materials
|   |   |-- module_c.py      ASR transcription + tashkeel
|   |   |-- module_d.py      Evaluation, feedback, sessions, adaptive params
|   |   |-- module_library.py  UN Digital Library search + fetch
|   |   |-- alignment.py     WhisperX forced alignment + LLM analysis
|   |   |-- chat.py          Chat endpoint
|   |   `-- auth.py          Login and signup
|   |-- services/
|   |   `-- llm_service.py   LLM provider abstraction (Gemini / Groq / local Aya)
|   |-- utils/               Groq client, RAG retrieval, UN documents, Arabic utils
|   `-- tests/               pytest tests
|-- Frontend/
|   |-- src/
|   |   |-- App.jsx          All React components + UI strings (EN/AR/FR)
|   |   `-- api.js           All fetch helpers (auth token in the JSON body)
|   |-- styles/
|   |   |-- main.css
|   |   `-- rtl.css
|   `-- index.html
`-- docs/
    |-- USER_GUIDE.md
    |-- ETIB_User_Guide.pdf
    `-- ETIB_Technical_Documentation.pdf
```

Module A — Speech Generation (Technical Notes)

RAG pipeline (document-grounded generation)

- Source document chunked: 1 800 characters per chunk, 250-character overlap
- Each chunk embedded with paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers)
- Cosine similarity computed between topic/query and all chunks
- Top-4 chunks injected into the LLM prompt as grounding context

UN grounding

- Searches UN Digital Library MARCXML API — tries up to 5 results in 3 languages (EN/FR/ES)
- Downloads the document PDF with curl and extracts its text
- 20-second per-PDF timeout; minimum 40 words to count as a valid document
- Falls back to ungrounded generation with factual accuracy rules if no document found

Arabic specifics

- All digit sequences converted to Eastern Arabic-Indic numerals
- LLM prompt instructs the model to use correct Arabic numeral and grammatical forms

Endpoints

```
POST /api/module-a/generate
POST /api/module-a/from-document
POST /api/module-a/retrieve-document-context
```

Module C — ASR Transcription (Technical Notes)

- Primary: Groq hosted whisper-large-v3 (~3 seconds per recording)
- Fallback: local faster-whisper, medium model, ~1-3 minutes on CPU
- Disfluency priming prompts per language — preserves hesitation markers (euh, um, yani)
- Arabic tashkeel added post-transcription: reflects student pronunciation including errors
- WhisperX forced alignment available when installed: provides word-level confidence scores

Endpoints

```
POST /api/module-c/transcribe
POST /api/module-c/align
GET  /api/module-c/status
```

Module D — Evaluation (Technical Notes)

Error detection

- Hesitations: regex matching per language (euh, um, ah, yani, ...)
- Number errors: number token comparison between source and student transcript
- Omissions: silence gaps > 500 ms in student audio flagged as possible omissions
- Terminology: key terms from source checked for presence in transcript

Adaptive parameters

- Recomputed after each session from the full score history
- Tracks problems_to_work_on and top_errors fields across all sessions
- Recommends difficulty, length, and domain for the next session

Session storage

- Stored in MongoDB collection: etib_interpreter_trainer.sessions
- Falls back to in-memory dict if MongoDB is unavailable (resets on server restart)
- Sessions keyed by user_id, or "anonymous" if the user is not authenticated

Endpoints

```
POST /api/module-d/full-evaluation
POST /api/module-d/evaluate
POST /api/module-d/tashkeel-compare
POST /api/module-d/pronunciation
POST /api/module-d/feedback
GET  /api/module-d/sessions
GET  /api/module-d/adaptive-params
```

Authentication Endpoints

Endpoint	Method	Description
/api/auth/signup	POST	Create a new user account
/api/auth/login	POST	Log in with email and password
/api/auth/me	GET	Check current session state
/api/auth/logout	POST	End the session (clear cookie)
/api/auth/validate-groq-key	POST	Legacy: test a Groq API key (no longer used by the UI)

Authentication uses a token sent in the JSON body of each request. MongoDB stores user accounts when available; an in-memory dict is used otherwise. Seed accounts (student@etib.edu, instructor@etib.edu) are created on first run.

Standards Compliance

Standard	How it applies
GDPR	User data minimised; only speech and transcript content is sent to the AI providers
ISO/IEC 27001	API keys kept in server environment variables, never in the repository or browser
IEEE 12207	Git version control with feature branches; modular service architecture
ISO 9241	Consistent UI design; full RTL (right-to-left) support for Arabic